



Bangladesh Army University of Science and Technology Khulna

Lab Report

Course Title : Data Structures and Algorithms-II Sessional
Course Code : CSE-216
Date of Submission : 25-Feb-2026
Experiment No : 01
Experiment Name : Implementations of BST

Submitted by

Name: Safwan Ahmad

ID: 1072420005101001

Department: CSE

Level: 02

Term: 02

Semester: January 2026

Submitted to

Name: Syed Nahin Hossain

Designation: Lecturer

Department: CSE

Institute: BAUST Khulna

Remark

Signature

Title : Implementation of Binary Search Tree

Objective :

- To understand the concept of BST.
- To implement insertion, searching, deletion, and traversal operations in BST using C++.
- To analyze how BST maintain sorted order.
- To understand recursive implementation of tree operations.

Introduction :

A tree is a non-linear data structure that represents hierarchical relationship. A special type of tree called BST which follows a specific rule:

- Left subtree contains values smaller than the root.
- Right subtree contains values greater than the root.
- Both left and right subtrees are also BSTs.

BST is widely used because it allows efficient searching, insertion, and deletion operations.

Theory :

Binary Tree :

A Binary Tree is a tree where each node has at most two children:

- left child
- right child

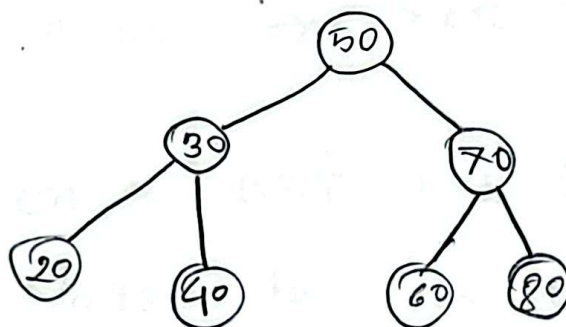
Binary search Tree (BST) property :

For every node:

- All nodes in the left subtree are smaller.
- All nodes in the right subtree are larger.

BST Example Diagram :

For inserted values : 50, 30, 70, 20, 40, 60, 80

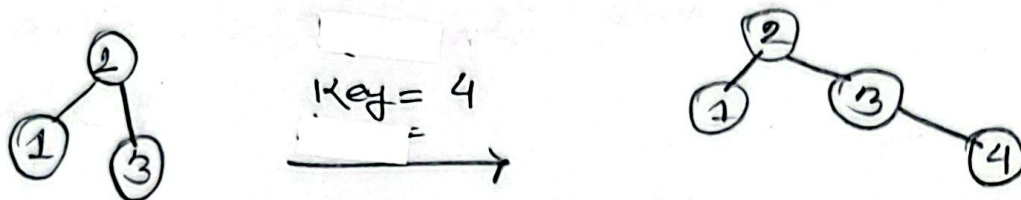


Operations in BST

Insertion :

- compare value with root
- If smaller $\rightarrow$ go left
- If larger $\rightarrow$ go right
- Insert at NULL position

Example :



Search :

- compare key with root.
- Traverse left or right accordingly
- If found $\rightarrow$ return true
- If NULL $\rightarrow$ not found

Deletion :

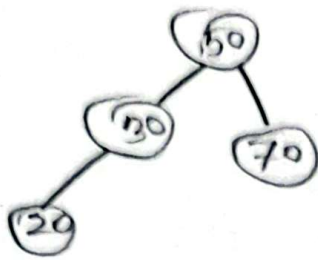
Deletion in a BST has three cases :

Case 1 : Node has no child (Leaf Node)

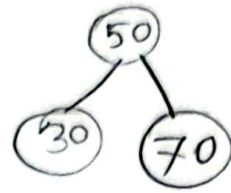
If the node to be deleted is a leaf node, simply remove it.

(4)

Example:



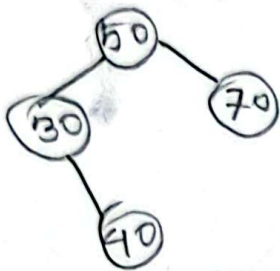
target = 20



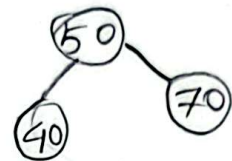
Case 2: Node has one child

If the node has one child, replace the node with its child.

Example:



target = 30

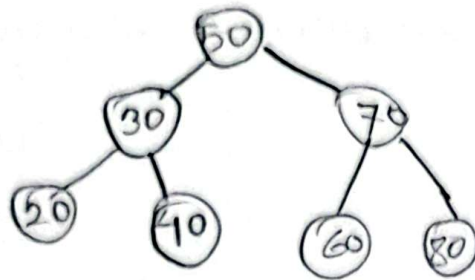


Case 3: Node has Two children

If the node has two children:

- Find the inorder ~~to~~ successor (smaller value in right subtree).
- Replace the node's value with that value.
- Delete the inorder successor.

Example :

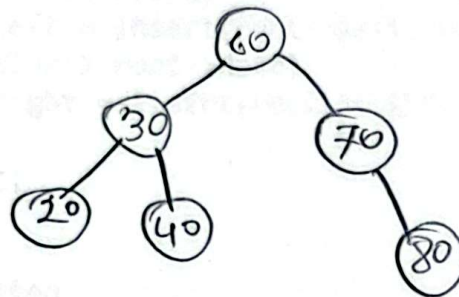


Step 1 : Inorder successor of 50 is 60
(minimum in right subtree)

Step 2 : Replace 50 with 60

Step 3 : Delete original 60

After deletion :



Here, 50 is replaced by 60 and original 60 node is removed.

Traversals :

- Inorder (Left $\rightarrow$ Root $\rightarrow$ Right)
- Preorder (Root $\rightarrow$ Left $\rightarrow$ Right)
- Postorder (Left $\rightarrow$ Right $\rightarrow$ Root)

Source Code

```
#include <iostream>
using namespace std;

// Node class (only for creating node)
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = NULL;
        right = NULL;
    }
};

// Insert function
Node* insert(Node* root, int value) {
    if (root == NULL)
        return new Node(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

// Search function
bool search(Node* root, int key) {
    if (root == NULL)
        return false;

    if (root->data == key)
        return true;
    else if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

// Find minimum value (used in deletion)
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
}
```

```
        return root;
    }

    // Delete function
    Node* deleteNode(Node* root, int key) {
        if (root == NULL)
            return NULL;

        if (key < root->data)
            root->left = deleteNode(root->left, key);
        else if (key > root->data)
            root->right = deleteNode(root->right, key);
        else {
            // Case 1: No child
            if (root->left == NULL && root->right == NULL) {
                delete root;
                return NULL;
            }

            // Case 2: One child
            else if (root->left == NULL) {
                Node* temp = root->right;
                delete root;
                return temp;
            }
            else if (root->right == NULL) {
                Node* temp = root->left;
                delete root;
                return temp;
            }

            // Case 3: Two children
            Node* temp = findMin(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }

        return root;
    }

    // Inorder Traversal
    void inorder(Node* root) {
        if (root == NULL)
            return;

        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
```

```
// Preorder Traversal
void preorder(Node* root) {
    if (root == NULL)
        return;

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

// Postorder Traversal
void postorder(Node* root) {
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}

// Main function
int main() {
    Node* root = NULL;

    // Insert values
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    cout << "Inorder: ";
    inorder(root);
    cout << endl;

    cout << "Preorder: ";
    preorder(root);
    cout << endl;

    cout << "Postorder: ";
    postorder(root);
    cout << endl;

    int key = 40;
    if (search(root, key))
        cout << key << " Found\n";
}
```

```
else
    cout << key << " Not Found\n";

    root = deleteNode(root, 30);

    cout << "After deleting 30 (Inorder): ";
    inorder(root);
    cout << endl;

    return 0;
}
```

Output:

```
PS D:\DSA II> cd "d:\DSA II\" ; if ($?) { g++ BST.cpp -o BST } ; if ($?) { .\BST }
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
40 Found
After deleting 30 (Inorder): 20 40 50 60 70 80
PS D:\DSA II>
```

Real Life Applications of BST:

BST is used in:

- Database indexing
- File system hierarchy
- Dictionary implementation
- Searching applications
- Maintaining sorted data dynamically
- Result searching with student ID.
- Voter list management with voter ID

Conclusion:

In this experiment, we successfully implemented a BST using C++. We performed insertion, searching, deletion and traversal operations. The BST maintains sorted order and provides efficient operations with average time complexity of $O(\log n)$.

This lab helped in understanding recursive tree operations and memory management in dynamic data structure.